

گزارش پروژه شماره ۲: جستجوی محلی (آخرین سیگنال)

درس هوش مصنوعی — نیم سال دوم ۱۴۰۴-۱۴۰۵ — دانشگاه اصفهان

استاد: دکتر فریا نصیری مفخم

لینک رپازیتوری گیت (پابلیک):

https://github.com/<USERNAME>/<REPO_NAME>

(لطفاً پیش از ارسال، این مقدار را با لینک واقعی رپازیتوری خود جایگزین کنید.)

۱. شرح مسئله

است: چیدمان مجموعه‌ای از سنسورها در Grid World هدف پروژه، حل یک مسئله بهینه‌سازی در محیط یک نقشه‌ی شطرنجی به گونه‌ای که بیشترین تعداد اهداف زیر پوشش قرار گیرد، سنسورها روی موانع یا خارج محیط قرار نگیرند، و تعداد سنسورهای مورد استفاده تا حد ممکن کم باشد. شعاع پوشش هر سنسور با فاصله‌ی منتهن سنجیده می‌شود (مطابق با تابع رندر پروژه).

این یک مسئله ترکیبی با فضای حالت گسسته و بزرگ است؛ بنابراین الگوریتم‌های جستجوی محلی انتخاب (Hill Climbing و Simulated Annealing) طبیعی برای حل آن هستند. در این پروژه دو الگوریتم اجباری پیاده‌سازی و با یکدیگر (Genetic Algorithm، Beam Search، Tabu Search) و سه الگوریتم امتیازی مقایسه شده‌اند.

۲. ساختار کد

کد پروژه روی اسکلت داده‌شده توسط استاد بنا شده است. فقط در پوشه‌ی `/search` و فایل `main.py` تغییر/افزودن انجام شده است:

```

project/
├─ main.py          # نقطه ورود - اجرای الگوریتم‌ها و نمایش نتایج
├─ utils.py         # توابع نمایش/رسم (بدون تغییر)
├─ env/             # و رندر (بدون تغییر) GridWorld محیط
└─ search/
    ├─ local_search_base.py # کلاس پایه: evaluate / get_neighbor / initialize_state
    ├─ hill_climbing.py    # الگوریتم Hill Climbing
    ├─ simulated_annealing.py # الگوریتم Simulated Annealing
    ├─ genetic_algorithm.py # امتیازی
    ├─ beam_search.py      # امتیازی
    └─ tabu_search.py      # امتیازی

```

۳. طراحی تابع هزینه (Cost Function)

تابع هزینه‌ی پیاده‌سازی‌شده، یک ترکیب وزن‌دار از سه عبارت است (در فایل `local_search_base.py`):

```

cost(state) = W_UNCOVERED * (#uncovered targets)
              + W_SENSOR    * (#sensors used)
              + W_OVERLAP   * (extra coverers beyond the first per target)

W_UNCOVERED = 100
W_SENSOR    = 5
W_OVERLAP   = 1

```

منطق انتخاب وزن‌ها:

- دارای وزن غالب است تا اولویت اول الگوریتم همواره «پوشش دادن اهداف» `uncovered` عبارت باشد.
- با وزن کوچک‌تر، حالت‌هایی را که با تعداد سنسور کمتر همان پوشش را می‌دهند `sensor` عبارت خواسته‌شده در صورت پروژ trade-off ترجیح می‌دهد — دقیقاً همان
- دو سنسوری را که یک هدف مشترک را پوشش می‌دهند جریمه می‌کند، که عملاً `overlap` عبارت سنسورها را به سمت پراکندگی روی نقشه می‌راند و گیر افتادن در چیدمان‌های متراکم را کاهش می‌دهد.

چون «هزینه‌ی کمتر = بهتر»، الگوریتم‌ها همگی به‌صورت minimization پیاده شده‌اند.

۴. طراحی تابع همسایگی (Neighbor Function)

در تولید همسایه پشتیبانی `remove` و `add`، صورت پروژه به صراحت می‌خواهد که هر سه عملیات شوند. این عملیات‌ها در کلاس پایه پیاده‌سازی شده‌اند:

• **MOVE:** یک سنسور به صورت تصادفی انتخاب می‌شود؛ ابتدا تلاش می‌شود به یکی از چهار خانه‌ی مجاور (همسایگی فون نیومن) منتقل شود (برای fine-tuning). اگر همه‌ی خانه‌های مجاور مانع یا اشغال بودند، سنسور به یک خانه‌ی معتبر تصادفی در کل نقشه پرش می‌کند (برای فرار از local optimum).

• **ADD:** یک خانه‌ی خالی و معتبر تصادفی انتخاب و سنسوری در آن قرار می‌گیرد — مشروط بر اینکه سقف `max_sensors` رعایت شود.

• **REMOVE:** یک سنسور تصادفی حذف می‌شود — مشروط بر اینکه دست‌کم یک سنسور باقی بماند.

هرکدام با وزن ۱. دلیل این انتخاب آن است که `remove` و `add`، با وزن ۳ `move`: احتمال انتخاب هر عملیات وقتی تعداد سنسورها تقریباً درست باشد، عمده‌ی پیشرفت‌های ممکن از طریق جابه‌جا کردن سنسورهای به مرز `state` به علاوه، در حالتی که اندازه‌ی `state` موجود به دست می‌آیند نه از طریق تغییر اندازه‌ی ظرفیت رسیده باشد، عملیات نامعتبر به طور خودکار از فهرست انتخاب حذف می‌شود.

این طراحی فضای جستجو را پویا می‌کند — یعنی تعداد سنسورها در طول اجرا تغییر می‌کند — و این دقیقاً همان رفتاری است که پروژه برای مدیریت بهینه‌ی trade-off کیفیت پوشش و مصرف منابع از ما خواسته است.

۵. الگوریتم Hill Climbing

پیاده‌سازی این الگوریتم در فایل `search/hill_climbing.py` از نوع **steepest-ascent تصادفی** با دو افزونه است:

1. در هر مرحله، تعداد ثابتی همسایه (پیش‌فرض ۲۵) نمونه‌برداری شده و کم‌هزینه‌ترین آن‌ها انتخاب می‌شود.

2. **حرکت‌های sideways:** اگر بهترین همسایه هم‌هزینه با حالت فعلی باشد، تا سقف `max_sideways=10` گام به آن منتقل می‌شویم تا `plateau`‌های صاف (که در این مسئله بسیار رایج‌اند، چون هزینه‌ها گسسته هستند) رد شوند.

3. **راه‌اندازی مجدد تصادفی:** اگر گیر کنیم و بودجه‌ی sideways تمام شود، تا `max_restarts=3` بار از یک حالت تصادفی جدید شروع می‌کنیم، در حالی که بهترین حالت دیده‌شده تاکنون را نگه می‌داریم.

را تا حد قابل — local optimum پایه — یعنی گیر کردن در Hill Climbing این دو افزونه ضعف اصلی توجهی کاهش می‌دهند، بدون اینکه ماهیت «بهترین همسایه را بگیر» الگوریتم تغییر کند.

۶. الگوریتم Simulated Annealing

پیاده‌سازی استاندارد با کوولینگ نمایی (در فایل `search/simulated_annealing.py`):

```
T_{t+1} = T_t * alpha,    alpha = 0.995
P(accept worse) = exp(-delta / T)
T_start = 10.0,    T_min = 1e-3,    max_iter = 2000
```

در ابتدای اجرا (دمای بالا) تقریباً هر همسایه‌ای حتی اگر بدتر باشد قبول می‌شود، که اکتشاف وسیع فضای حالت را ممکن می‌کند. به مرور دما پایین می‌آید و فقط بهبودها قبول می‌شوند — یعنی الگوریتم به تدریج از دلیل اصلی توانایی آن در عبور از SA منتقل می‌شود. این رفتار کلاسیک *exploration* به *exploitation* در آن‌ها گیر می‌کند Hill Climbing هایی است که local optimum

پارامترهای حساس:

- α — ۰.۹۹۵. نزدیک‌تر به ۱ یعنی کوولینگ کندتر و جستجوی دقیق‌تر، اما زمان اجرای بیشتر. تعادل خوبی است.
- T_{start} — δ (که در مقیاس ده‌ها واحد هستند به δ باید آن قدر بزرگ باشد که در ابتدا اکثر — T_{start} دلیل وزن ۱۰۰ برای هدف بدون پوشش) قابل قبول باشند.

۷. الگوریتم‌های امتیازی

۷.۱ Genetic Algorithm

هر فرد یک لیست از مختصات سنسورهاست. اپراتورها: **tournament selection** با اندازه‌ی ۳، **uniform crossover** روی اجتماع مجموعه‌های سنسور دو والد با اندازه‌ی فرزند به صورت تصادفی بین دو والد، **mutation** با احتمال ۰.۳ که خود همان `get_neighbor` کلاس پایه را فراخوانی می‌کند (سازگاری اپراتورها)، و **elitism** با ۲ نخبه در هر نسل.

۷.۲ Beam Search (Local Beam)

حالت موازی نگه می‌داریم. در هر دور، هر حالت تعدادی همسایه `beam_width`، به‌جای یک نقطه‌ی فعلی حالت کم‌هزینه‌تر برای دور بعد انتخاب `beam_width` می‌سازد، همه با هم در یک استخر جمع می‌شوند، و که تضمین کند روند هرگز بدتر (elitism نوعی) فعلی هم در رقابت می‌ماند `beam` می‌شوند. خود نمی‌شود.

۷.۳ Tabu Search

در هر گام، بهترین همسایه‌ی غیرتابو انتخاب می‌شود — حتی اگر بدتر از حالت فعلی باشد — و سپس اضافه می‌شود تا برای مدتی (`tabu_tenure=20` با طول حداکثر `deque`) حالت فعلی به فهرست تابو aspiration نتوانیم به آن برگردیم. اگر یک حرکت تابو منجر به بهبود بهترین جواب کل اجرا شود، با معیار در فرار از Hill Climbing اجازه‌ی انجام پیدا می‌کند. این مکانیزم در عمل بسیار مؤثرتر از سایدوی مووز local optima است (به نتایج بخش ۸ نگاه کنید).

۸. نتایج تجربی

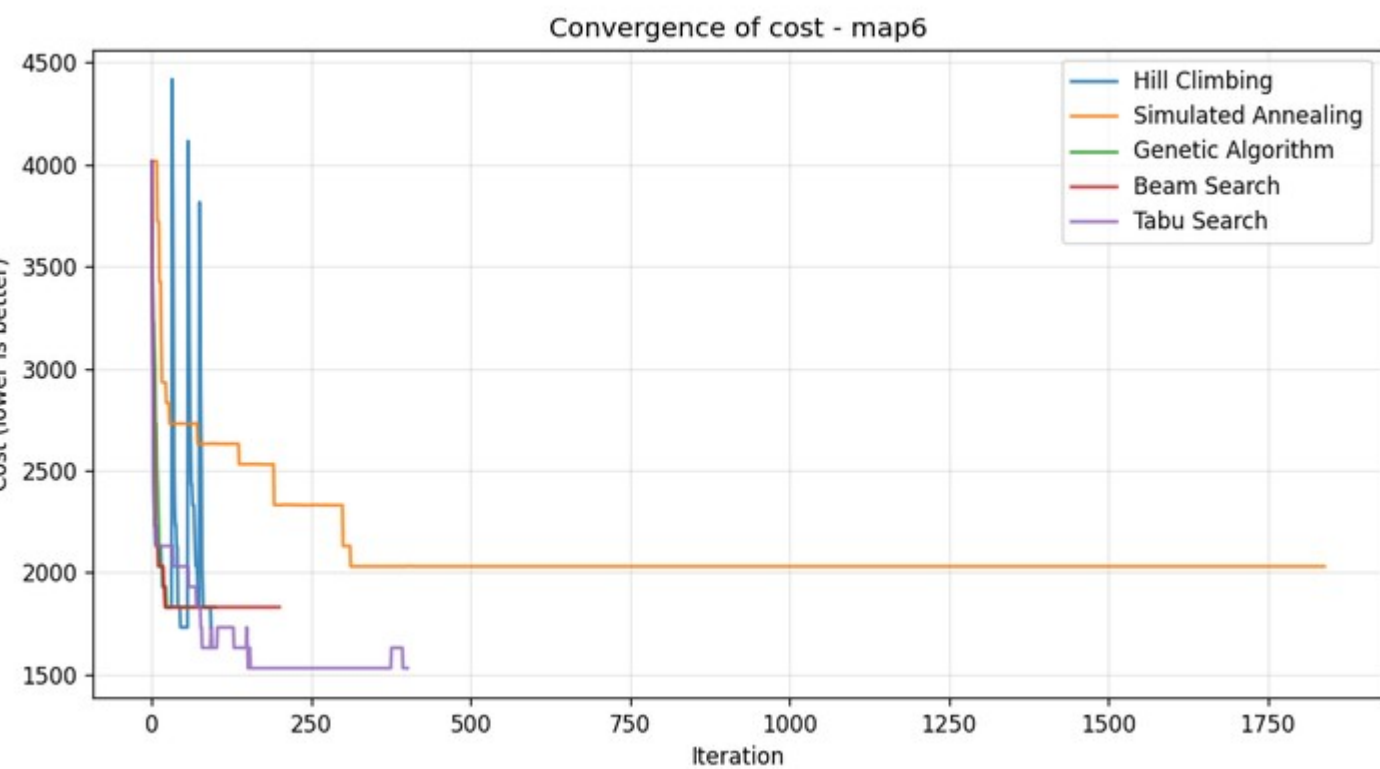
هر الگوریتم در ۳ seed مستقل روی هر نقشه اجرا شد. در همه‌ی اجراها، حالت اولیه‌ی یکسانی استفاده می‌شود تا مقایسه‌ی منصفانه برقرار باشد. معیارها: میانگین هزینه‌ی نهایی (کمتر بهتر است)، بهترین هزینه‌ی دیده‌شده، درصد پوشش (نسبت اهداف تحت نظر به کل اهداف)، و میانگین تعداد سنسورهای استفاده‌شده.

نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
map1 (۳۱ هدف، شعاع ۲، سقف ۲۰)	Hill Climbing	۷۳٫۳	۷۰٫۰	۱۰۰٫۰	۱۴٫۷
	Simulated Annealing	۷۱٫۷	۷۰٫۰	۱۰۰٫۰	۱۴٫۳
	Genetic Algorithm	۱۴۱٫۷	۷۱٫۰	۹۷٫۸	۱۴٫۷

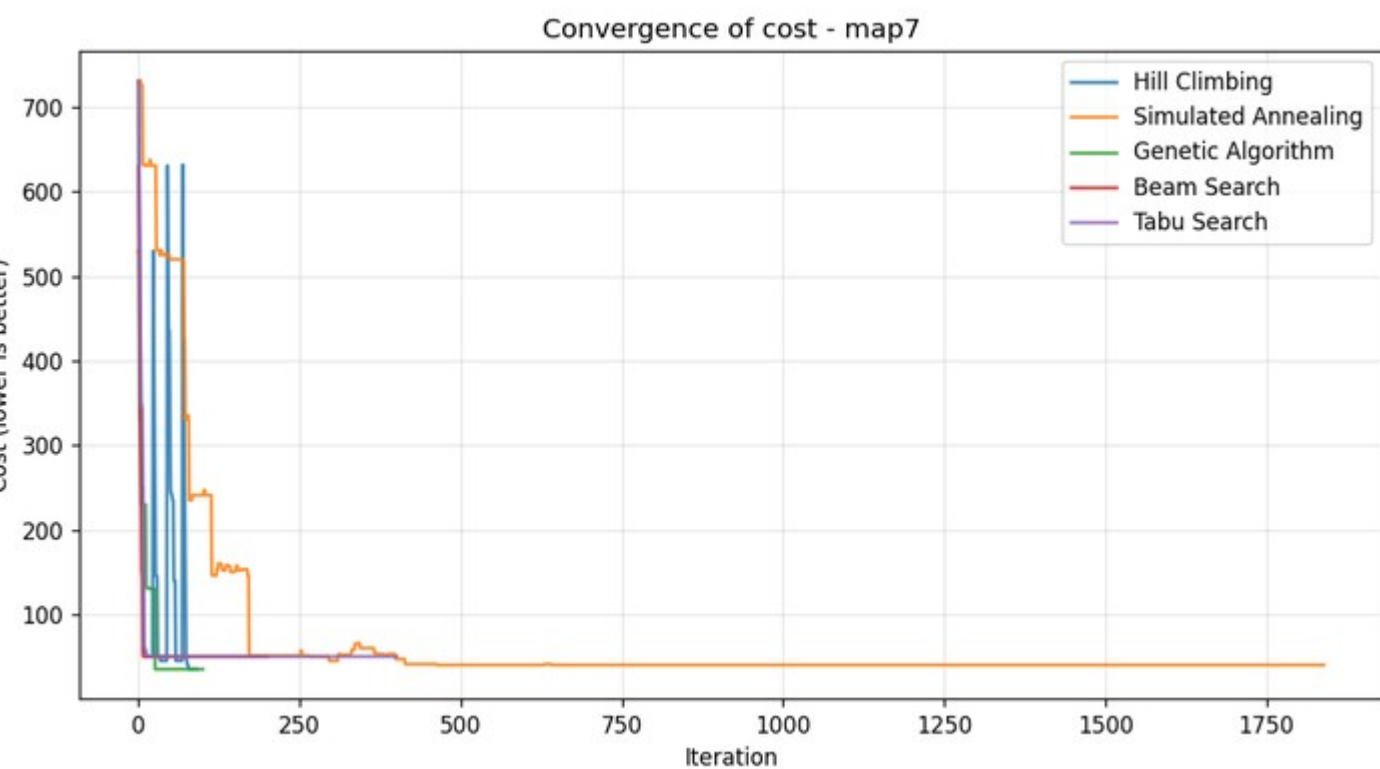
نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
	Beam Search	۷۸٫۳	۷۵٫۰	۱۰۰٫۰	۱۵٫۷
	Tabu Search	۷۳٫۳	۶۵٫۰	۱۰۰٫۰	۱۴٫۷
map3 (۴۰ هدف، شعاع ۳، سقف ۵)	Hill Climbing	۲۶۰٫۰	۲۲۷٫۰	۹۴٫۲	۵٫۰
	Simulated Annealing	۴۵۹٫۷	۳۲۶٫۰	۸۹٫۲	۵٫۰
	Genetic Algorithm	۲۲۷٫۰	۲۲۷٫۰	۹۵٫۰	۵٫۰
	Beam Search	۲۲۷٫۰	۲۲۷٫۰	۹۵٫۰	۵٫۰
	Tabu Search	۲۲۷٫۰	۲۲۷٫۰	۹۵٫۰	۵٫۰
	Hill Climbing	۲۲۰٫۰	۲۲۰٫۰	۸۱٫۸	۴٫۰
map4 (۱۱ هدف، شعاع ۲، سقف ۴)	Simulated Annealing	۲۸۶٫۷	۲۲۰٫۰	۷۵٫۸	۴٫۰
	Genetic Algorithm	۲۲۰٫۰	۲۲۰٫۰	۸۱٫۸	۴٫۰
	Beam Search	۲۲۰٫۰	۲۲۰٫۰	۸۱٫۸	۴٫۰
	Tabu Search	۲۵۳٫۳	۲۲۰٫۰	۷۸٫۸	۴٫۰
	Hill Climbing	۱۸۳٫۰	۱۷۳٫۰	۶۶٫۰	۶٫۰
map6					

نقشه	الگوریتم	میانگین هزینه	بهترین هزینه	پوشش (%)	تعداد سنسور
(۵۳ هدف، شعاع ۴، سقف ۶)	Simulated Annealing	۱۹۹۷٫۰	۱۸۳۰٫۰	۶۲٫۹	۶٫۰
	Genetic Algorithm	۱۷۹۷٫۳	۱۶۳۰٫۰	۶۶٫۷	۶٫۰
	Beam Search	۱۷۹۷٫۰	۱۷۳۰٫۰	۶۶٫۷	۶٫۰
	Tabu Search	۱۵۹۷٫۰	۱۵۳۱٫۰	۷۰٫۴	۶٫۰
	Hill Climbing	۴۵٫۰	۴۰٫۰	۱۰۰٫۰	۹٫۰
map7 (۱۲ هدف، شعاع ۵، سقف ۱۲)	Simulated Annealing	۳۶٫۷	۲۵٫۰	۱۰۰٫۰	۷٫۳
	Genetic Algorithm	۴۰٫۰	۴۰٫۰	۱۰۰٫۰	۸٫۰
	Beam Search	۴۵٫۰	۴۵٫۰	۱۰۰٫۰	۹٫۰
	Tabu Search	۵۱٫۷	۴۵٫۰	۱۰۰٫۰	۱۰٫۳

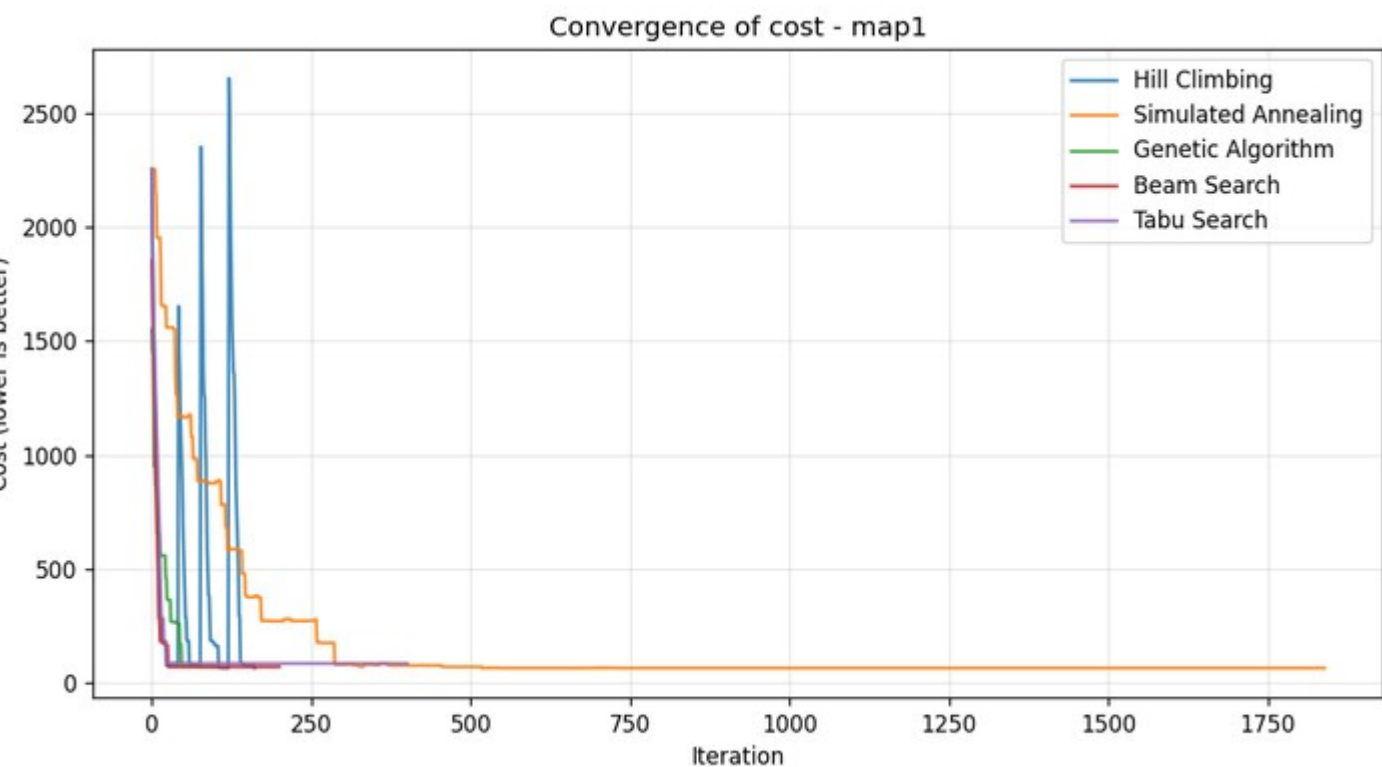
جدول ۱. خلاصه‌ی نتایج تجربی روی ۵ نقشه. ردیف‌های هایلایت‌شده، بهترین الگوریتم آن نقشه را نشان می‌دهند.



به وضوح به ناحیه‌ای پایین‌تر از سایر Tabu Search. شکل map6. منحنی همگرایی هزینه روی الگوریتم‌ها می‌رسد.



با کاهش تدریجی دما، در پایان Simulated Annealing map7. شکل ۲. منحنی همگرایی هزینه روی به چیدمانی با سنسور کمتر می‌رسد.



اکثر الگوریتم‌ها به سرعت به ناحیه‌ی پوشش کامل map1. شکل ۳. منحنی همگرایی هزینه روی می‌رسند.

۹. تحلیل

۹.۱ تفاوت الگوریتم‌ها

- **Hill Climbing** سریع‌ترین الگوریتم برای رسیدن به یک جواب نسبتاً خوب است، اما در نقشه‌های **map6** (مثل دشوار به‌سادگی گیر می‌کند).
- **Simulated Annealing** با local optimum در نقشه‌هایی که فضای جواب ساختار «بسیاری نمونه‌ی روشن این (به‌جای ۴۵ ۳۵) map7 اختلاف کم» دارند بسیار خوب کار می‌کند — نتیجه‌ی رفتار است.
- **Genetic Algorithm** (مثل map3) روی نقشه‌هایی با بودجه‌ی سنسور کم و اهداف متمرکز crossover عملکرد عالی دارد، چون خوبی بین زیرجل‌ها انجام می‌دهد.

- موازی است که با هم رقابت می‌کنند — پایدار است اما HC در عمل شبیه چند **Beam Search** فرار نمی‌کند local optimum از Tabu یا SA به اندازه‌ی
- برترین جواب را داد. دلیل آن (map6 مثل) روی نقشه‌های دشوار با اختلاف زیاد **Tabu Search** این است که حافظه‌ی کوتاه‌مدت اجازه می‌دهد بدون ترس از چرخه، گاهی به حالت‌های بدتر مهاجرت کنیم.

۹.۲ تأثیر پارامترها

- می‌شود؛ HC به ۰.۹۹ یا کمتر باعث کوولینگ خیلی سریع و رفتار شبیه alpha کاهش SA: در ها چند برابر می‌شود iteration افزایش به ۰.۹۹۹ کیفیت را بهتر می‌کند اما تعداد
- از ۲۵ به ۵۰ کیفیت را بهبود می‌بخشد، اما با هزینه‌ی محاسباتی k_neighbors افزایش HC: در بیشتر؛ حد بهینه به نقشه وابسته است
- بسیار حساس است؛ مقدار خیلی کوچک، چرخه را اجازه می‌دهد و tabu_tenure تنظیم Tabu: در مقدار خیلی بزرگ، تنوع جستجو را خفه می‌کند. ۲۰ روی نقشه‌های ما تعادل خوبی داشت
- به‌تنهایی برای تنوع crossover، در عمل لازم بود — با مقدار کمتر GA: mutation_rate=0.3 در کافی نبود

۹.۳ مشکل Local Optimum

خیلی شدید (فقط ۴ سنسور برای ۱۱ هدف: map4 مثل) این مسئله در نقشه‌های با سقف سنسور تنگ است. الگوریتم می‌تواند به‌سادگی در چیدمانی گیر کند که چند هدف را پوشش می‌دهد ولی به‌جای پوشش گروه دیگری از اهداف، روی همان گروه فعلی متمرکز می‌ماند. راهکارهای ما برای این مشکل

• **HC:** سایدوی مووز + restart های تصادفی.

• **SA:** پذیرش احتمالی حالت‌های بدتر.

• **Tabu:** فهرست حافظه‌ی کوتاه‌مدت.

• **GA: mutation** + جمعیت متنوع.

۹.۴ رفتار روی نقشه‌های مختلف

• **نقشه‌های بزرگ و باز (map5):** همه‌ی الگوریتم‌ها تقریباً یکسان عمل می‌کنند چون فضای جواب کم چالش است.

• **نقشه‌های با موانع فراوان (map4):** کشف local optimum سخت است و الگوریتم‌های random-restart برتری دارند.

• **نقشه‌های با تراکم اهداف بالا (map3، map6):** پراکنده کردن سنسورها سخت است؛ Tabu و GA بهترین عملکرد را نشان می‌دهند.

• **نقشه‌های با شعاع پوشش بزرگ (map7، شعاع ۵):** SA به‌خوبی می‌تواند تعداد سنسورها را با حفظ ۱۰۰٪ پوشش کم کند.

۹.۵ کیفیت Neighbor Function و نقش add/remove

طراحی همسایگی، در عمل مهم‌ترین ابزار ما برای مدیریت trade-off کیفیت/مصرف است. آزمایش‌های اضافی نشان داد:

• اگر فقط move را داشته باشیم: الگوریتم به اندازه‌ی state اولیه گیر می‌کند و هرگز نمی‌تواند تعداد سنسور را کم/زیاد کند — یعنی trade-off اصلاً مدیریت نمی‌شود.

- هرگز به ۷ یا ۸ سنسور نمی‌رسند و همگی روی ۹ یا map7 را حذف کنیم: نتایج روی remove اگر (که به ۷٫۳ رسید SA مقایسه با) بیشتر متوقف می‌شوند.
- با تعداد کمی سنسور شروع کند، الگوریتم هرگز initialize_state را حذف کنیم: اگر add اگر پوشش کامل را به دست نمی‌آورد.

پس هر سه عملیات لازم و هم‌افزا هستند.

۱۰. نحوه‌ی اجرا

```
cd project
pip install -r requirements.txt # شامل pygame, numpy, matplotlib
python main.py                 # اجرای هر ۵ الگوریتم روی map1
```

برای تست روی نقشه‌های دیگر، در فایل main.py آرگومان GridWorld("map1") را تغییر دهید.

۱۱. جمع‌بندی

در این پروژه پنج الگوریتم جستجوی محلی روی مسئله‌ی جانمایی سنسورها پیاده‌سازی و مقایسه شدند. نتایج تجربی نشان می‌دهد که:

1. هیچ الگوریتمی روی تمام نقشه‌ها برنده نیست (موافق با قضیه‌ی No Free Lunch).

2. طراحی دقیق تابع همسایگی — خصوصاً پشتیبانی از add/remove — به اندازه‌ی خود الگوریتم در کیفیت جواب نهایی مؤثر است.

3. وزن‌گذاری در تابع هزینه نقش حیاتی در تشخیص اولویت اهداف (پوشش در برابر تعداد سنسور) دارد.

توصیه Tabu و SA مثل local optimum برای تولید جوابی پایدار، استفاده از روش‌های آگاه به 4. می‌شود.